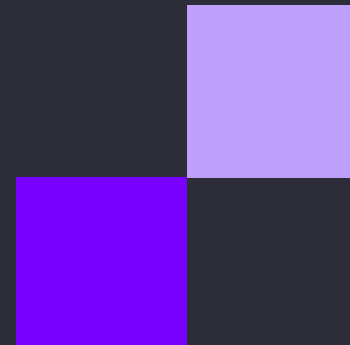




онлайн
университет



ВЫСШАЯ ШКОЛА
ЭКОНОМИКИ

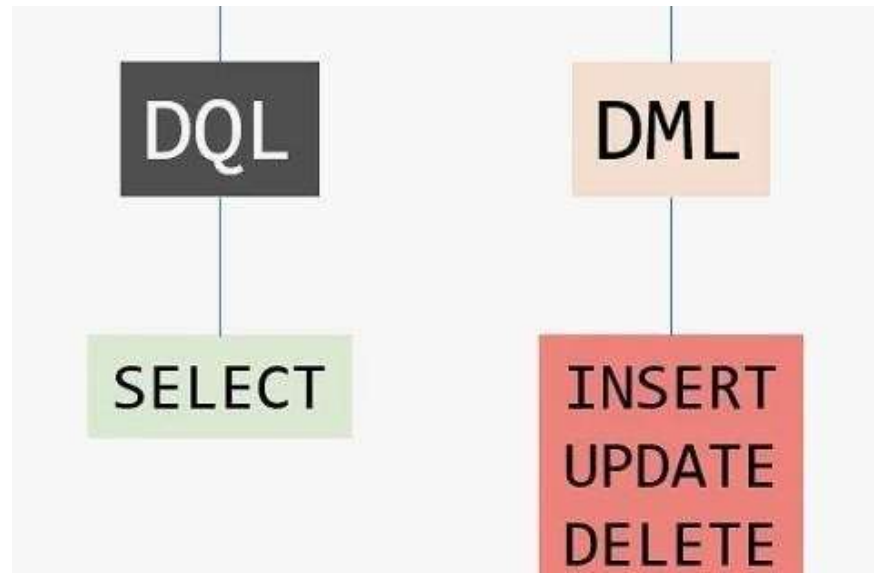


SQL для начинающих

Занятие 11



DML (Data Manipulation Language) и DQL (Data Query Language)



- SELECT – извлекает данные из базы данных
- INSERT – вставляет новые данные в таблицу
- UPDATE – изменяет существующие данные в таблице
- DELETE – удаляет данные из таблицы

DDL (Data Definition Language)

Язык определения данных. DDL включает команды, которые используются для определения или изменения структуры данных и её объектов. Таких как таблицы, индексы, схемы и т.д. Ключевые команды DDL:

- CREATE – используется для создания новых объектов в базе данных.
Например, таблиц или индексов
- ALTER – позволяет изменять структуру существующих объектов
- DROP – удаляет объекты из базы данных
- TRUNCATE – быстро удаляет все строки из таблицы, освобождая место, занимаемое этими строками

Создание таблиц

Для создания таблицы в SQL используется команда CREATE TABLE, за которой следует имя новой таблицы и определение столбцов, включая их типы данных и, при необходимости, другие атрибуты (например, ограничения)

```
CREATE TABLE имя_таблицы (  
    имя_столбца1 тип данных_столбца1 ограничения,  
    имя_столбца2 тип данных_столбца2 ограничения,  
    ...  
);
```

Создание таблиц

Допустим, мы хотим создать таблицу students с информацией о студентах университета. Таблица должна содержать уникальный идентификатор студента (ID), его имя, возраст и специальность

```
CREATE TABLE students(  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    field VARCHAR(100)  
);
```

Создание таблиц

Если таблицу нужно создать в существующей схеме, то схема добавляется в название таблицы:

```
CREATE TABLE hse.students(  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    field VARCHAR(100)  
);
```

Без указания названия схемы таблица создается в схеме public

Создание таблиц

Допустим, мы хотим создать таблицу students с информацией о студентах университета. Таблица должна содержать уникальный идентификатор студента (ID), его имя, возраст и специальность

```
CREATE TABLE students(  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    field VARCHAR(100)  
);
```

В этом примере:

- id – столбец с целочисленным типом данных, который автоматически увеличивает свое значение при добавлении новых строк (SERIAL) и который является первичным ключом (PRIMARY KEY) таблицы. Первичный ключ уникально идентифицирует каждую строку в таблице
- name и field – это столбцы с текстовым типом данных (VARCHAR), где 100 указывает максимальную длину строки
- age – столбец с целочисленным типом данных

Ограничения

В SQL можно (и даже зачастую нужно) указывать различные ограничения для столбцов таблицы, чтобы обеспечить точность и надежность данных. Некоторые из наиболее часто используемых ограничений:

- PRIMARY KEY – указывает столбец как первичный ключ таблицы
- NOT NULL – запрещает NULL (пустые) значения в таблице
- UNIQUE – гарантирует, что все значения в столбце уникальны
- CHECK – устанавливает диапазон допустимых значений для поля или столбца
- DEFAULT – устанавливает значение по умолчанию для столбца, если значение не указано при вставке данных

Primary Key

С помощью выражения **PRIMARY KEY** столбец можно сделать первичным ключом.

```
CREATE TABLE students(  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    field VARCHAR(100)  
);
```

Primary Key

Первичный ключ может быть составным (compound key). Такой ключ может потребоваться, если у нас сразу два столбца должны уникально идентифицировать строку в таблице. Например:

```
CREATE TABLE students(  
    id SERIAL,  
    email VARCHAR(100),  
    name VARCHAR(100),  
    age INT,  
    field VARCHAR(100),  
    PRIMARY KEY(id, email)  
);
```

NOT NULL

Чтобы указать, может ли столбец принимать значение NULL, при определении столбца ему можно задать атрибут **NULL** или **NOT NULL**. Если этот атрибут явным образом не будет использован, то по умолчанию столбец будет допускать значение NULL. Исключением является тот случай, когда столбец выступает в роли первичного ключа - в этом случае по умолчанию столбец имеет значение NOT NULL.

```
CREATE TABLE students(  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    age INT NOT NULL,  
    field VARCHAR(100)  
);
```

UNIQUE

Если мы хотим, чтобы столбец имел только уникальные значения, то для него можно определить атрибут **UNIQUE**.

```
CREATE TABLE students(  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    age INT NOT NULL UNIQUE,  
    field VARCHAR(100)  
);
```

UNIQUE

Также его можно определить на уровне таблицы:

```
CREATE TABLE students(  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    age INT NOT NULL,  
    field VARCHAR(100),  
    UNIQUE(name,age)  
);
```

CHECK

Ключевое слово **CHECK** задает ограничение для диапазона значений, которые могут храниться в столбце. Для этого после слова CHECK указывается в скобках условие, которому должен соответствовать столбец или несколько столбцов. Например, возраст студентов не может быть меньше 0 или больше 100:

```
CREATE TABLE students(  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    age INT NOT NULL CHECK (age > 0 AND age < 100),  
    field VARCHAR(100),  
    UNIQUE(name, age)  
);
```

DEFAULT

DEFAULT определяет значение по умолчанию для столбца. Если при добавлении данных для столбца не будет предусмотрено значение, то для него будет использоваться значение по умолчанию.

```
CREATE TABLE students(  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    age INT NOT NULL CHECK (age > 0 AND age < 100) DEFAULT 18,  
    field VARCHAR(100),  
    UNIQUE(name, age)  
);
```

Добавление значений в таблицу

Для заполнения таблицы `students`, которую мы создали ранее, можно использовать команду `INSERT INTO`. Давайте добавим несколько записей в эту таблицу. Первичный ключ заполнится автоматически.

```
INSERT INTO students(name, age, field)
VALUES ('Александр', 19, 'Информатика');
```

```
INSERT INTO students(name, age, field)
VALUES ('Мария', 20, 'Математика');
```

```
INSERT INTO students(name, age, field)
VALUES ('Иван', 18, 'Физика');
```


Добавление значений в таблицу

Можно одним запросом добавить несколько записей:

```
INSERT INTO students(name, age, field)
VALUES
('Елена', 21, 'Биология'),
('Николай', 22, 'Химия'),
('Ольга', 20, 'Литературоведение');
```

Изменение значений

```
UPDATE students  
SET field = 'Математика'  
WHERE id = 3
```

Удаление значений

```
DELETE FROM students  
WHERE id = 3
```

Рекомендую никогда не забывать условие WHERE, иначе удалятся все записи из таблицы.

Практика! Задача 1

Создайте таблицу clients с данными о клиентах Ростелекома.

Поля таблицы:

- client_id — идентификатор пользователя, первичный ключ, тип integer, значения в столбце должны быть уникальны, а само поле должно быть заполнено
- firstname — имя пользователя, тип text, поле должно быть заполнено
- lastname — фамилия пользователя, тип text
- auth_type — тип авторизации: text

Практика! Задача 1

```
CREATE TABLE clients (  
  user_id INTEGER UNIQUE NOT NULL,  
  firstname TEXT NOT NULL,  
  lastname TEXT,  
  auth_type TEXT  
);
```

Практика! Задача 2

Создайте таблицу `projects` с данными о проектах компании.

Поля таблицы:

- `project_id` — идентификатор проекта, первичный ключ, тип `integer`.
- `manager` - `varchar` длины 50, имя менеджера, отвечающий за проект
- `project_name` — название проекта, тип `text`, поле обязательно для заполнения.
- `start_date` — дата начала проекта, тип `date`, поле обязательно для заполнения.
- `end_date` — дата окончания проекта, тип `date`, не может быть раньше начала.

Комбинация из `manager` и `project_name` должна быть уникальной.

Практика! Задача 2

```
CREATE TABLE projects (  
    project_id SERIAL PRIMARY KEY,  
    manager VARCHAR(50),  
    project_name TEXT NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE CHECK (end_date >= start_date),  
    UNIQUE (manager, project_name)  
);
```

Практика! Задача 3

Создайте таблицу `members` для учета членов клуба программы лояльности.

Поля таблицы:

- `member_id` — идентификатор члена, первичный ключ, тип `integer`.
- `full_name` — полное имя, тип `text`, поле обязательно для заполнения.
- `join_date` — дата вступления, тип `date`, не может быть в будущем.
- `membership_type` — тип членства, тип `text`, допустимые значения: `'standard'`, `'premium'`, `'lifetime'`.
- `email` — email, тип `text`, уникальный.

Практика! Задача 3

```
CREATE TABLE members (  
    member_id SERIAL PRIMARY KEY,  
    full_name TEXT NOT NULL,  
    join_date DATE CHECK (join_date <= CURRENT_DATE),  
    membership_type TEXT CHECK (membership_type IN ('standard', 'premium', 'lifetime')),  
    email TEXT UNIQUE );
```